

CSC490 Assignment A5: Testing Your Project

Chau Nguyen (nguy2829)

Avni Agrawal (agraw264)

Rishit Dagli (dagliris)

Date: March 28, 2026

1 Profiling Execution Time

We profiled five important functions before and after the performance changes using Python's standard library `profile` module. The baseline measurements come from `profiling/important_functions/summary.json`, and the improved measurements come from `profiling/important_functions_improved/summary.json`. Both runs used the local mock LLM path and the heuristic critic so that the comparison reflects code changes rather than external service latency.

Function	Calls	Before cumulative.	After cumulative.	Before total	After total
<code>run_vertical_inference</code>	150	0.970212	0.587898	0.009680	0.006919
<code>SceneHypothesisLoop.train</code>	1	0.184729	0.161943	0.004125	0.002918
<code>train_vertical_slice</code>	1	0.258973	0.136619	0.000049	0.000586
<code>_build_heuristic_score</code>	5000	0.059346	0.058551	0.024567	0.024475
<code>MockSceneLLM.generate_scene_program</code>	2500	0.130528	0.046277	0.031871	0.015770

Table 1: Before-versus-after profiling results for the five selected functions.

`run_vertical_inference`

- This function was chosen because it is the main inference entry point and covers bank loading, hypothesis selection, scene generation, scoring, and artifact writing, so it represents the full inference path seen by users.
- It was slow mainly because it repeatedly serialized JSON payloads, created directories, and wrote several output artifacts for every inference run.
- The implementation change reused serialized payloads such as the experiment config, critic score, selected hypothesis IDs, and scene program, and it also switched to a cached compact JSON writer that avoids repeated directory creation work.
- The cumulative time improved from 0.970212 seconds to 0.587898 seconds, which is about a 39.4% reduction, although artifact I/O is still the main remaining bottleneck.

`SceneHypothesisLoop.train`

- This function was chosen because it is the core hypothesis training loop and therefore the main learning-time hot path in the system.
- It was slow mainly because it interleaved model work with frequent output generation, including per-program JSON writes, traces, failure logs, metrics, and snapshots.

- The implementation change buffered per-program outputs until after the hot loop and reused already serialized prediction, trace, and failure rows instead of recomputing them for JSONL output.
- The cumulative time improved from 0.184729 seconds to 0.161943 seconds, which is about a 12.3% reduction, showing that reducing repeated serialization helped even though persistence still dominates.

`train_vertical_slice`

- This function was chosen because it is the top-level training entry point exposed through the CLI and therefore represents the full orchestration path for a training run.
- It was slow mostly because it repeatedly serialized the experiment configuration and passed duplicated payloads into lower-level orchestration steps.
- The implementation change computed the serialized experiment configuration once and reused it across loop construction, manifest creation, and storage mirroring.
- The cumulative time improved from 0.258973 seconds to 0.136619 seconds, which is about a 47.2% reduction, confirming that cutting repeated orchestration overhead had a noticeable effect.

`_build_heuristic_score`

- This function was chosen because every generated scene is scored with it, so it directly affects both evaluation throughput and the feedback loop during training.
- It was slow because it repeatedly rebuilt supported-asset sets and prompt-mentioned asset sets across many scoring calls, even when the room type or prompt was unchanged.
- The implementation change added caches for supported asset sets and mentioned asset sets, and it avoided unnecessary sorting work unless unsupported assets were actually present.
- The cumulative time improved from 0.059346 seconds to 0.058551 seconds, which is about a 1.3% reduction, so this function was already relatively efficient and only yielded a small gain.

`MockSceneLLM.generate_scene_program`

- This function was chosen because scene generation sits directly on the hot path for both training and inference, and it is executed many times across the full pipeline.
- It was slow because it repeatedly parsed prompt assets, scanned hypotheses against supported assets, and recomputed style and requested-count information across many similar calls.
- The implementation change added caching for style extraction, requested counts, supported assets, prompt assets, and hypothesis-asset scans, and reused a stable tuple form of the selected hypotheses.
- The cumulative time improved from 0.130528 seconds to 0.046277 seconds, which is about a 64.5% reduction, making it the largest improvement among the five functions.

Overall, the comparison shows that the changes helped most in scene generation and orchestration, while heuristic scoring changed very little. The remaining dominant bottleneck is still artifact I/O, so the next round of optimization should focus on writing fewer files and avoiding unnecessary serialization in the training and inference pipelines.

2 Showing Code Coverage

Figure 1 and shows that our tests achieve 100 coverage across all files. Figure 2–5 show how our tests achieve 100 coverage across all classes.

Coverage report: 100%

Files Functions Classes

coverage.py v7.13.5, created at 2026-03-27 21:20 -0400

File ▲	statements	missing	excluded	coverage
src\compos3d__init__.py	0	0	0	100%
src\compos3d_stages.py	4	0	0	100%
src\compos3d\app_config.py	44	0	0	100%
src\compos3d\aws_runtime.py	299	0	0	100%
src\compos3d\catalog.py	41	0	0	100%
src\compos3d\cli.py	77	0	0	100%
src\compos3d\compute__init__.py	2	0	0	100%
src\compos3d\compute\ec2_runner.py	122	0	0	100%
src\compos3d\config.py	89	0	0	100%
src\compos3d\data__init__.py	2	0	0	100%
src\compos3d\data\dataset.py	7	0	0	100%
src\compos3d\data\spatiallm.py	167	0	2	100%
src\compos3d\evaluation__init__.py	1	0	0	100%
src\compos3d\evaluation\critic.py	142	0	0	100%
src\compos3d\evaluation\service.py	10	0	0	100%
src\compos3d\hypothesis__init__.py	1	0	0	100%
src\compos3d\hypothesis\engine.py	344	0	2	100%
src\compos3d\hypothesis\loop.py	378	0	0	100%
src\compos3d\hypothesis\service.py	55	0	2	100%
src\compos3d\hypothesis\training_logging.py	252	0	0	100%
src\compos3d\llm__init__.py	3	0	0	100%
src\compos3d\llm\bedrock.py	27	0	0	100%
src\compos3d\llm\scene_llm.py	221	0	0	100%
src\compos3d\models.py	155	0	0	100%
src\compos3d\procedural__init__.py	2	0	0	100%
src\compos3d\procedural\layout.py	122	0	0	100%
src\compos3d\procedural\runner.py	49	0	0	100%
src\compos3d\procedural\service.py	99	0	0	100%
src\compos3d\schemas__init__.py	2	0	0	100%
src\compos3d\schemas\manifest.py	61	0	0	100%
src\compos3d\storage__init__.py	13	0	2	100%
src\compos3d\storage\local.py	30	0	0	100%
src\compos3d\storage\multibucket_s3.py	66	0	0	100%
src\compos3d\storage\paths.py	26	0	0	100%
Total	2913	0	8	100%

coverage.py v7.13.5, created at 2026-03-27 21:20 -0400

Figure 1: File-level coverage report showing code coverage across all files.

Coverage report: 100%

Files Functions Classes

coverage.py v7.13.5, created at 2026-03-27 21:20 -0400

File ▲	class	statements	missing	excluded	coverage
src\compos3d__init__.py	(no class)	0	0	0	100%
src\compos3d_stages.py	StagePendingError	0	0	0	100%
src\compos3d_stages.py	(no class)	4	0	0	100%
src\compos3d\app_config.py	AppConfig	0	0	0	100%
src\compos3d\app_config.py	(no class)	44	0	0	100%
src\compos3d\aws_runtime.py	SecretBinding	0	0	0	100%
src\compos3d\aws_runtime.py	RuntimePaths	0	0	0	100%
src\compos3d\aws_runtime.py	_IMDSv2	23	0	0	100%
src\compos3d\aws_runtime.py	CheckpointSyncManager	33	0	0	100%
src\compos3d\aws_runtime.py	(no class)	243	0	0	100%
src\compos3d\catalog.py	(no class)	41	0	0	100%
src\compos3d\cli.py	(no class)	77	0	0	100%
src\compos3d\compute__init__.py	(no class)	2	0	0	100%
src\compos3d\compute\ec2_runner.py	EC2JobSpec	0	0	0	100%
src\compos3d\compute\ec2_runner.py	EC2JobRunner	89	0	0	100%
src\compos3d\compute\ec2_runner.py	(no class)	33	0	0	100%
src\compos3d\config.py	GeneratorConfig	0	0	0	100%
src\compos3d\config.py	CriticConfig	0	0	0	100%
src\compos3d\config.py	TrainingConfig	0	0	0	100%
src\compos3d\config.py	RenderConfig	0	0	0	100%
src\compos3d\config.py	LoggingConfig	0	0	0	100%
src\compos3d\config.py	ExperimentConfig	0	0	0	100%
src\compos3d\config.py	(no class)	89	0	0	100%

Figure 2: Class-level coverage report (part 1 of 4).

src\compos3d\data__init__.py	(no class)	2	0	0	100%
src\compos3d\data\dataset.py	(no class)	7	0	0	100%
src\compos3d\data\spatiallm.py	SpatialLMCandidate	0	0	0	100%
src\compos3d\data\spatiallm.py	(no class)	167	0	2	100%
src\compos3d\evaluation__init__.py	(no class)	1	0	0	100%
src\compos3d\evaluation\critic.py	CriticUnavailableError	0	0	0	100%
src\compos3d\evaluation\critic.py	_CriticPayload	0	0	0	100%
src\compos3d\evaluation\critic.py	HeuristicSceneCritic	3	0	0	100%
src\compos3d\evaluation\critic.py	BedrockVLMCritic	34	0	0	100%
src\compos3d\evaluation\critic.py	(no class)	105	0	0	100%
src\compos3d\evaluation\service.py	EvaluateRequest	0	0	0	100%
src\compos3d\evaluation\service.py	(no class)	10	0	0	100%
src\compos3d\hypothesis__init__.py	(no class)	1	0	0	100%
src\compos3d\hypothesis\engine.py	(no class)	344	0	2	100%
src\compos3d\hypothesis\loop.py	HypothesisLoopConfig	0	0	0	100%
src\compos3d\hypothesis\loop.py	RenderArtifacts	0	0	0	100%
src\compos3d\hypothesis\loop.py	SceneEvaluation	0	0	0	100%
src\compos3d\hypothesis\loop.py	SceneHypothesisLoop	293	0	0	100%
src\compos3d\hypothesis\loop.py	(no class)	85	0	0	100%
src\compos3d\hypothesis\service.py	TrainingRequest	0	0	0	100%
src\compos3d\hypothesis\service.py	InferenceRequest	0	0	0	100%
src\compos3d\hypothesis\service.py	(no class)	55	0	2	100%
src\compos3d\hypothesis\training_logging.py	TrainingLogger	189	0	0	100%
src\compos3d\hypothesis\training_logging.py	(no class)	63	0	0	100%
src\compos3d\llm__init__.py	(no class)	3	0	0	100%
src\compos3d\llm\bedrock.py	BedrockConfig	1	0	0	100%
src\compos3d\llm\bedrock.py	BedrockChatClient	10	0	0	100%
src\compos3d\llm\bedrock.py	(no class)	16	0	0	100%

Figure 3: Class-level coverage report (part 2 of 4).

src\compos3d\llm\bedrock.py	(no class)	16	0	0	100%
src\compos3d\llm\scene_llm.py	LLMUnavailableError	0	0	0	100%
src\compos3d\llm\scene_llm.py	StructuredOutputError	0	0	0	100%
src\compos3d\llm\scene_llm.py	MockSceneLLM	22	0	0	100%
src\compos3d\llm\scene_llm.py	BedrockSceneLLM	35	0	0	100%
src\compos3d\llm\scene_llm.py	(no class)	164	0	0	100%
src\compos3d\models.py	AssetSpec	0	0	0	100%
src\compos3d\models.py	ConstraintSpec	0	0	0	100%
src\compos3d\models.py	RenderSpec	0	0	0	100%
src\compos3d\models.py	SceneProgram	0	0	0	100%
src\compos3d\models.py	TrainingExample	0	0	0	100%
src\compos3d\models.py	TrainingDataset	0	0	0	100%
src\compos3d\models.py	HypothesisRecord	0	0	0	100%
src\compos3d\models.py	CriticScore	0	0	0	100%
src\compos3d\models.py	PredictionRecord	0	0	0	100%
src\compos3d\models.py	HypothesisEvaluationRecord	0	0	0	100%
src\compos3d\models.py	FailureRecord	0	0	0	100%
src\compos3d\models.py	TrainingTraceRecord	0	0	0	100%
src\compos3d\models.py	EvaluationSummary	0	0	0	100%
src\compos3d\models.py	(no class)	155	0	0	100%
src\compos3d\procedural__init__.py	(no class)	2	0	0	100%
src\compos3d\procedural\layout.py	(no class)	122	0	0	100%
src\compos3d\procedural\runner.py	(no class)	49	0	0	100%
src\compos3d\procedural\service.py	BackendSmokeRequest	0	0	0	100%
src\compos3d\procedural\service.py	ReferenceGenerationRequest	0	0	0	100%
src\compos3d\procedural\service.py	BuildSceneRequest	0	0	0	100%
src\compos3d\procedural\service.py	(no class)	99	0	0	100%

Figure 4: Class-level coverage report (part 3 of 4).

src\compos3d\schemas__init__.py	(no class)	2	0	0	100%
src\compos3d\schemas\manifest.py	GitInfo	0	0	0	100%
src\compos3d\schemas\manifest.py	RunManifest	0	0	0	100%
src\compos3d\schemas\manifest.py	(no class)	61	0	0	100%
src\compos3d\storage__init__.py	(no class)	13	0	2	100%
src\compos3d\storage\local.py	LocalStore	19	0	0	100%
src\compos3d\storage\local.py	(no class)	11	0	0	100%
src\compos3d\storage\multibucket_s3.py	MultiLayerS3Store	52	0	0	100%
src\compos3d\storage\multibucket_s3.py	(no class)	14	0	0	100%
src\compos3d\storage\paths.py	LakePaths	0	0	0	100%
src\compos3d\storage\paths.py	(no class)	26	0	0	100%
Total		2913	0	8	100%

Figure 5: Class-level coverage report (part 4 of 4).

3 Test Coverage Breadth

Here is a detailed description of each of our integration tests and the need for each test:

1. **tests/test_blender_rendering.py** This file is an integration suite for real Blender-backed rendering via `build_scene` (guarded by `pytest.importorskip("bpy")`). It validates that scene programs produce real rendered image artifacts and that manifests include expected metadata under different room presets and seeds. These tests are necessary because render pipelines involve external runtime dependencies and filesystem side effects that unit tests cannot faithfully simulate.
 - **test_build_scene_renders_dining_room:** This test builds a dining-room scene from a minimal scene program and asserts rendered views exist, are non-empty, and are discoverable via manifest output. It verifies baseline render viability for one canonical room type. This is necessary because it directly checks that procedural generation reaches tangible image outputs.
 - **test_build_scene_renders_living_room:** This test repeats render validation for living-room input, ensuring room-type variation still yields successful view generation. It validates coverage across scene categories. This is necessary to avoid hidden room-specific regressions in asset spawning/placement logic.
 - **test_build_scene_manifest:** This test asserts manifest keys (`rendered_views`, `elapsed_seconds`) exist and that elapsed time is numeric and positive. It validates runtime metadata contract from rendering service. This is necessary because orchestration and reporting code consume these fields for observability and diagnostics.
 - **test_build_scene_different_seeds_produce_variation:** This test runs rendering twice with different seeds and asserts both runs complete with rendered outputs. It validates seed-path stability under repeated execution. This is necessary because non-deterministic procedural branches must still remain operational across seed changes.
2. **tests/test_direct_render.py** This file is an integration test for cross-stage handoff from inference output to rendering input. It runs mock inference to produce `scene_program.json`,

then immediately feeds that artifact into `build_scene` and validates rendered outputs. These tests are necessary because they verify interface compatibility between modules, not just each module in isolation.

- **test_direct_render_full_pipeline:** This test executes two connected phases: `run_vertical_inference` (mock provider, no render) to create `scene_program.json`; `build_scene` with that file to render views. It validates existence and non-empty size of produced images. This is necessary because many production failures occur at stage boundaries (schema/path assumptions), and this test explicitly checks that inference artifacts are consumable by the renderer end-to-end.
3. **tests/test_generation_pipeline.py** This file validates the generation/inference pipeline from both unit and integration perspectives. It checks generator initialization, scene program generation contracts, scoring range consistency, API surface stability, room-type adherence, and one end-to-end inference execution using a frozen hypothesis bank. These tests are necessary because this file sits at a critical junction of ranking, LLM output normalization, and critic evaluation; regressions here can silently degrade output quality while still producing syntactically valid artifacts.
- **test_generator_initialization:** This test verifies that constructing the generator with `GeneratorConfig(provider="mock")` yields the expected provider identity. It ensures configuration-driven provider resolution is stable. This is necessary because provider mis-resolution can route requests to unintended code paths.
 - **test_generator_blender_code_generation:** This test verifies that mock generation returns a valid `SceneProgram` with a requested room type and non-empty assets for a simple prompt. It checks output shape and minimum semantic completeness. This is necessary to guarantee downstream stages receive structurally useful scene programs.
 - **test_generator_scene_evaluation:** This test generates a scene and evaluates it with the heuristic critic, asserting all score dimensions remain in $[0, 1]$. It validates score normalization and bounded metric semantics. This is necessary because downstream aggregation and comparisons assume normalized score ranges.
 - **test_generator_api:** This test confirms required generator interface methods are present and callable (`generate_scene_program`, `generate_hypotheses`). It is an API contract check. This is necessary to prevent interface drift that breaks orchestration code relying on these methods.
 - **test_generator_respects_room_type:** This test iterates over supported room types and verifies generated outputs preserve the requested room type. It validates explicit room-conditioning behavior. This is necessary because room-type drift directly undermines controllability guarantees.
 - **test_generation_end_to_end:** This integration test runs `run_vertical_inference` over a frozen bank and verifies core output artifacts (`scene_program_path`, `critic_score_path`, selected hypotheses), scene room type, and critic score range. It validates composed behavior across bank loading, selection, generation, and evaluation. This is necessary to catch cross-module regressions that unit tests cannot expose.
4. **tests/test_gold_aggregation_distributed.py** This file validates Gold-layer publication and aggregation semantics, including empty-prefix behavior, final-bank publication, training summary persistence, score aggregation correctness, and Gold population after training with store

mirroring. These tests are necessary because Gold is the final consumer-facing layer; errors here directly affect what downstream users or services treat as canonical outputs.

- **test_gold_aggregation_empty_silver:** This test asserts listing a non-existent Gold prefix returns []. It validates graceful handling of sparse/uninitialized storage states. This is necessary for robust tooling that scans optional experiment outputs.
 - **test_gold_write_latest_bank:** This test writes a representative final bank to latest.json in Gold and reads it back, validating field fidelity (including numeric values like reward). It verifies publication of canonical hypothesis artifacts. This is necessary because inference consumers often depend on latest.json as the authoritative bank.
 - **test_gold_write_training_summary:** This test writes and reloads training_summary.json, checking experiment identity and metric content persistence. It validates run-level summary publication. This is necessary for auditability and quick run comparison without re-reading full traces.
 - **test_gold_aggregation_creates_statistics:** This test constructs prediction records with critic scores and validates aggregate_prediction_scores emits a well-formed summary with expected counts and bounded averages. It verifies aggregation math and schema consistency. This is necessary because Gold summaries are used for reporting and model-quality monitoring.
 - **test_gold_populated_after_training_with_store:** This integration test runs train_vertical_slice with a LocalStore, then verifies Gold receives expected artifacts (latest.json, training summary). It validates training-to-lake mirroring end-to-end for the Gold layer. This is necessary to ensure production-like training runs actually publish final outputs.
5. **tests/test_graceful_error_handling.py** This module focuses on graceful failure behavior across multiple subsystems (LLM parsing, critic parsing, storage IO, and evaluation entry-points). The goal is not to test the correct path, but to ensure common real-world failure modes produce explicit exceptions with useful messages instead of cryptic tracebacks, partial artifacts, or silent misbehavior. These tests are necessary because this project relies on external systems (LLM providers, file-based artifact layouts, optional rendered images), and failures at those boundaries are common; the code needs predictable, diagnosable failure contracts for both CLI use and programmatic use.
- **test_scene_llm_extract_json_payload_malformed_raises_value_error:** This test calls the private JSON-extraction helper in compos3d.llm.scene_llm with a response string that contains no JSON object and asserts a ValueError with a clear “Could not locate JSON object” message. This is necessary because LLM responses can include formatting or malformed text; extraction must fail loudly and clearly so upstream callers can treat it as structured-output failure.
 - **test_critic_extract_json_payload_malformed_raises_value_error:** This test mirrors the previous one but for the critic-side JSON extraction helper in compos3d.evaluation.critic. It ensures malformed critic payloads raise ValueError with an informative message. This is necessary because critic parsing failures should be surfaced as actionable parsing errors rather than producing partial/incorrect scores.
 - **test_bedrock_scene_llm_credential_error_maps_to_unavailable:** This test monkey-patches the Bedrock client to raise a credential-expiration-like error (“session has expired”) and asserts it is mapped to LLMUnavailableError with reauthentication guid-

ance. This is necessary because credential failures are a frequent operational issue, and the code should provide a clear next step rather than a generic runtime error.

- **test_bedrock_scene_llm_invalid_json_maps_to_structured_output_error:** This test makes the Bedrock client return a response whose text is not JSON, and asserts a `StructuredOutputError` indicating invalid JSON output. This is necessary to distinguish “transport/provider is working” from “model returned an invalid schema,” which affects how the user/debugger should respond.
 - **test_local_store_read_missing_file_raises_file_not_found:** This test ensures `LocalStore.read_json` raises `FileNotFoundError` when asked to read a key that does not exist. This is necessary because the storage layer is used as a data-lake abstraction; missing artifacts should fail predictably rather than returning `None` or raising opaque errors later.
 - **test_evaluate_prediction_dir_missing_artifacts_raises_file_not_found:** This test calls `engine.evaluate_prediction_dir` on a non-existent predictions directory and asserts a `FileNotFoundError` mentioning `predictions.jsonl`. This is necessary because evaluation is artifact-driven; when inputs are missing, the system should tell you exactly what artifact is required.
 - **test_build_scene_llm_unsupported_provider_raises_value_error:** This test confirms `build_scene_llm` rejects unknown provider strings with a `ValueError`. This is necessary because provider selection is configuration-driven, and misconfiguration should fail immediately and clearly.
 - **test_evaluate_prediction_dir_fallback_manifest_without_critic_file_raises:** This test sets up a directory with an `inference_manifest.json` but without the required `critic_score.json`, then asserts `evaluate_prediction_dir` raises `FileNotFoundError`. This is necessary because the evaluation code has a fallback path that derives summary metrics from a manifest + critic score; the test ensures that fallback still fails explicitly when its required artifact is missing, rather than silently producing misleading outputs.
6. **tests/test_orchestrator.py** This file verifies orchestration/service-layer wiring for train, inference, and evaluation flows. It includes request construction checks, stage-level pipeline outputs, callable API checks, type import checks, and a full train to infer to evaluate integration run. These tests are necessary because orchestration is where independent subsystems are composed; wiring regressions frequently occur even when individual units pass in isolation.
- **test_orchestrator_initialization:** This test validates that `TrainingRequest` constructs with expected defaults/fields. It confirms request object schema and default behavior. This is necessary because orchestration correctness begins with stable request contracts.
 - **test_orchestrator_data_pipeline:** This test runs training and asserts key output artifacts exist (`hypothesis.bank.json`, `metrics.json`, `predictions.jsonl`). It validates core training pipeline materialization. This is necessary to ensure downstream stages have required artifacts.
 - **test_orchestrator_training_pipeline:** This test checks training result metadata (`num_predictions`, `run_dir`) and verifies the bank artifact exists. It validates result contract and artifact persistence. This is necessary for consumers of service return payloads.
 - **test_orchestrator_generation_pipeline:** This test trains, then runs inference from the produced bank, and validates scene output schema/room-type sanity. It verifies

cross-stage handoff from training output to inference input. This is necessary because inter-stage contract mismatches are a common integration failure mode.

- **test_orchestrator_api:** This test confirms main service entrypoints are importable and callable. It is an API stability check. This is necessary to catch accidental export/removal regressions.
 - **test_pipeline_types_enum:** This test verifies important model/type imports and basic constructability (SceneProgram, strategy-related symbols). It validates type-level interface continuity. This is necessary for compatibility with code that imports these symbols directly.
 - **test_orchestrator_full_system:** This integration test runs the full flow: train, infer, evaluate, then validates summary metrics and output file creation. It verifies end-to-end stage composition under the service layer. This is necessary as a high-confidence regression guard for the user-visible workflow.
7. **tests/test_pipeline_e2e.py** This file is a pure integration suite for the full Bronze → Silver → Gold lake lifecycle under local storage. It runs `train_vertical_slice` with mirroring enabled and validates that artifacts are produced in each layer, that key files are present, and that high-level layer status checks remain healthy. These tests are necessary because they verify cross-stage contracts that unit tests cannot: end-to-end artifact production, layer transitions, and lake consistency after real pipeline execution.
- **test_e2e_date_partition:** This test validates formatting of `utc_date_parts()` output (YYYY, MM, DD). It ensures date partition keys have stable canonical shape for path generation. This is necessary because partition formatting errors can break prefix scans and storage organization across the pipeline.
 - **test_e2e_storage_buckets:** This test executes a training run with LocalStore and asserts non-empty file presence across Bronze, Silver, and Gold prefixes. It validates that all three layers are populated by one complete run. This is necessary because lake mirroring is only useful if every layer receives expected artifacts.
 - **test_e2e_bronze_ingestion:** This test validates Bronze-specific outputs after training, including presence of run-level manifest under the run prefix. It checks raw-ingestion/provenance layer completeness. This is necessary because Bronze is the source-of-truth layer for replay and audit.
 - **test_e2e_silver_transformation:** This test validates Silver outputs, specifically presence of `hypothesis_bank.json` and `metrics.json`, and checks that bank payload shape is non-empty and structurally valid. It verifies validated intermediate artifact contracts. This is necessary because Gold aggregation and analytics depend on Silver correctness.
 - **test_e2e_gold_aggregation:** This test validates Gold publication after training by checking `latest.json` and `training_summary.json`, then confirming content sanity (non-empty bank, matching experiment metadata). It verifies final artifact promotion. This is necessary because downstream inference/reporting typically consume Gold-layer outputs directly.
 - **test_e2e_data_lake_status:** This test performs a full run and checks counts of files in Bronze/Silver/Gold are all ≥ 0 . It provides a coarse but robust health signal of the entire storage pipeline. This is necessary as a practical guardrail against systemic regressions that could leave a layer empty.

8. **tests/test_repair_regeneration_integration.py** This module is an integration-style stress test of the repair/regeneration branch inside the hypothesis training loop. It intentionally constructs a scenario where the critic always returns a score below the configured success threshold, forcing the loop to (a) record failures into the failure bank, (b) invoke the LLM in repair mode, (c) clear pending failure buffers, and (d) still finish the run while producing the expected output artifacts. This is necessary because the repair pathway is a “rare but critical” control-flow branch: it is only exercised under repeated failure conditions, yet it must remain stable to prevent long training runs from crashing, silently skipping recovery, or producing incomplete provenance logs.

- **test_repair_regeneration_path_writes_failure_artifacts_and_completes:** This test sets up a minimal TrainingDataset containing two bedroom examples, then runs SceneHypothesisLoop.train() with a custom critic (_AlwaysFailCritic) that always emits over-all=0.0. The test verifies that the training result reports at least one prediction and at least one regeneration event, that the LLM is actually called with focus=”repair” (via _RepairAwareLLM.focus_calls), and that pending.failure_examples[”bedroom”] is cleared after regeneration. It also checks that failed_scene_bank.jsonl exists and contains valid JSONL rows with the expected room_type, and that core run artifacts (predictions.jsonl, training_trace.jsonl, manifest.json) are still written despite continuous failure. This test is necessary because it validates the loop’s recovery behavior, logging/provenance guarantees, and “complete even under persistent failure” property—exactly the kinds of issues that can otherwise show up only after long executions.

9. **tests/test_silver_transformation_distributed.py** This file validates Silver-layer behavior for validated intermediate artifacts: missing-prefix resilience, hypothesis-bank and metrics writes, critic score persistence, data-quality clamping, and Silver population after training with mirroring. These tests are necessary because Silver is the curated layer many analyses and Gold publications rely on; schema/quality issues here can propagate into final reports.

- **test_silver_transformation_empty_bronze:** This test asserts missing Silver prefixes return an empty listing rather than errors. It validates graceful behavior for sparse states. This is necessary for robust pipeline scanners and rerun tooling.
- **test_silver_write_hypothesis_bank:** This test writes and reloads a hypothesis bank in Silver and checks content fidelity and cardinality. It validates persistence of validated bank artifacts. This is necessary because Silver serves as a stable intermediate checkpoint.
- **test_silver_write_metrics:** This test writes training metrics and verifies key numeric fields on reload. It validates metrics serialization integrity. This is necessary for trustworthy monitoring and progression tracking.
- **test_silver_write_critic_scores:** This test writes a critic score artifact and verifies existence via the store API. It validates per-scene score persistence in Silver. This is necessary because downstream evaluation/reporting depends on these intermediate score files.
- **test_silver_transformation_data_quality:** This test checks _clamp_unit behavior on normal and out-of-range inputs. It validates score normalization rules. This is necessary to enforce metric quality invariants before aggregation/publication.
- **test_silver_populated_after_training_with_store:** This integration test runs training with a local lake store and verifies Silver is populated with expected artifacts (notably

hypothesis bank). It validates training-to-silver mirroring in a production-like path. This is necessary to ensure Silver contracts hold in real pipeline execution, not just isolated writes.

10. **tests/test_aws_runtime.py** This is a unit test suite for the AWS runtime orchestration layer and checkpoint synchronization utilities. It validates S3-backed input hydration, Secrets Manager-based credential injection, runtime path, checkpoint restore/upload, and the top-level runtime entrypoint under mocked AWS clients and CLI dispatch.

- **test_hydrate_remote_inputs_downloads_supported_s3_paths:** This test validates that an S3 dataset URI is created for local storage and that the corresponding CLI argument is rewritten to the hydrated file path. This is necessary because downstream execution paths operate on filesystem artifacts.
- **test_hydrate_secrets_sets_environment:** This test verifies that secrets fetched from Secrets Manager are transformed into process-level environment variable bindings. It confirms that runtime initialization correctly injects credentials such as API keys into the execution environment. This is necessary because training, evaluation, and experiment logging subsystems depend on authenticated runtime state.
- **test_hydrate_secrets_skips_missing_optional_secret:** This test confirms that missing optional secrets do not trigger fatal startup failures and that resolvable secrets are still injected successfully. It verifies fault-tolerant behavior when development or staging environments contain only partially provisioned secret mappings. This is necessary because non-critical configuration gaps should not invalidate otherwise executable jobs.
- **test_resolve_runtime_paths_training_uses_checkpoint_prefix:** This test validates that a training command resolves both the local run directory and the corresponding remote S3 checkpoint prefix. It verifies the path-derivation invariants used by training output management and resume semantics. This is necessary because interrupted cloud jobs can only be resumed when checkpoint addressing is stable and reproducible.
- **test_checkpoint_sync_manager_restores_and_uploads:** This test verifies that the checkpoint manager can restore `resume_state.json` from S3 and subsequently upload newly produced local artifacts such as `metrics.json`. It validates both recovery semantics and periodic checkpoint synchronization behavior. This is necessary because long-running cloud jobs must persist intermediate state across preemption and interruption events.
- **test_runtime_main_hydrates_s3_inputs_and_writes_runtime_context:** This test exercises the main AWS runtime entrypoint with mocked config, S3, secrets, and inner CLI execution. It verifies that remote inputs are hydrated, command dispatch is preserved correctly, and `runtime_context.json` is emitted as provenance metadata. This is necessary because the runtime wrapper is responsible for coordinating configuration loading, secret hydration, CLI forwarding, and runtime context serialization.

11. **tests/test_runtime_and_engine_edge_cases.py** This is a unit test suite for edge conditions and fallback branches across the AWS runtime, hypothesis engine, training loop, training logger, EC2 runner, and procedural runner. It checks for graceful handling of errors and edge cases.

- **test_aws_runtime_cli_parsing_and_path_helpers:** This test validates CLI normalization, option parsing and replacement, S3 URI validation, runtime path resolution,

and the no-op branch of runtime-context serialization. It is necessary because small parsing or argument-rewrite defects in wrapper code can invalidate every downstream command invocation.

- **test_aws_runtime_s3_sync_edges_and_imds:** This test covers empty S3 prefixes, basic upload/download behavior, and multiple Instance Metadata Service success and failure branches. It is necessary because infrastructure code must fail gracefully under missing objects, metadata endpoint failures, and transient network faults.
- **test_checkpoint_manager_handlers_and_main_resume_branch:** This test checks checkpoint manager thread behavior, spot interruption detection, shutdown signal handling, and resuming after interruption.
- **test_aws_runtime_secret_hydration_and_inner_cli:** This test verifies secret-fetch failure behavior, partial-secret decoding behavior, inner CLI command construction, and the module `_main_` entrypoint path. It is necessary because runtime bootstrap logic depends on correct subprocess argument synthesis and predictable credential materialization behavior.
- **test_ec2_runner_remaining_simple_branches:** This test covers AMI discovery, subnet and security-group fallback behavior, GPU requirement checks. It is necessary because EC2 launch configuration contains multiple conditional branches whose failure modes can silently produce invalid infrastructure state.
- **test_engine_and_loop_remaining_branches:** This test covers engine lake mirroring, empty inference filtering, weighted-vote fallback behavior, config overrides that enable W&B, dataset mismatch rejection during resume, and empty-bank initialization. It is necessary because orchestration logic often fails at boundary conditions rather than during nominal training runs.
- **test_training_logging_remaining_branches:** This test covers missing `wandb` installation handling, disabled initial media logging, empty media payload behavior, table creation fallback, corrupt image handling for previews and animations, render-directory inference, and the `_mean([]) == 0.0` edge case. It is necessary because the logging subsystem interacts with optional dependencies and heterogeneous media inputs, so it should fail safely rather than terminate training.
- **test_procedural_runner_import_fallback:** This test verifies that the procedural runner falls back to the correct import and configuration root when the expected gin config path is unavailable. It is necessary because import-time path assumptions can invalidate rendering environments before any scene generation begins.

4 Test Coverage Depth

This section discusses the depth of our test coverage.

The following files contain our unit tests, and below is a detailed explanation of what each test covers.

1. **tests/test_bedrock.py**

- **test_resolve_bedrock_config_uses_env_defaults:** Verifies Bedrock configuration is derived from environment variables when explicit arguments are missing. Essential for deployed environments to prevent hardcoded fallback regressions and configuration drift.
 - **test_resolve_bedrock_config_explicit_values_override_env:** Validates that explicit function arguments take precedence over environment defaults, ensuring reproducible runs and controlled experiments.
 - **test_bedrock_chat_client_payload_construction:** Guards the API contract by ensuring the request payload matches the expected schema. Prevents runtime failures caused by outbound request shape mismatches.
 - **test_bedrock_chat_client_omits_optional_fields_when_not_given:** Confirms optional fields are omitted when not provided, reducing payload ambiguity and avoiding provider-side validation issues.
 - **test_bedrock_chat_client_propagates_runtime_client_error:** Ensures transport/runtime exceptions are propagated for observability and reliable error handling at higher layers.
 - **test_bedrock_vlm_response_parsing_and_clamping:** Checks that VLM critic responses are parsed correctly and numeric outputs are clamped to $[0, 1]$. Prevents invalid model-generated values from distorting evaluation metrics.
 - **test_bedrock_vlm_credential_error_maps_to_unavailable:** Ensures session-expiry failures map to `CriticUnavailableError` with actionable guidance, providing predictable operational messaging.
 - **test_bedrock_vlm_non_retryable_error_maps_to_unavailable_generic:** Verifies non-credential failures map to a consistent domain-level error to simplify caller-side exception logic.
2. **tests/test_bronze_ingestion.py** This file is a focused unit-level validation of Bronze-layer ingestion fundamentals using `LocalStore` and canonical Bronze key patterns. It verifies that raw scene artifacts can be written, retrieved, listed, and schema-validated at ingestion time. The tests are necessary because Bronze is the first persistence layer in the data lifecycle; defects here can propagate silently into Silver/Gold and compromise reproducibility.
- **test_bronze_write_and_read_scene:** This test writes a representative raw scene payload into a date-partitioned Bronze path, then reads it back and validates critical fields (`scene_id`, `room_type`). It confirms the basic write/read contract and that returned storage URIs are non-empty. This is necessary because ingestion correctness starts with preserving raw record fidelity.
 - **test_bronze_list_prefix:** This test writes multiple Bronze scene objects plus an unrelated Silver object, then asserts that `list_prefix` over the Bronze prefix returns exactly Bronze entries. It validates layer/prefix isolation and listing semantics. This is necessary to prevent cross-layer contamination during downstream ingestion scans.
 - **test_bronze_bytes_round_trip:** This test writes raw bytes (render-like binary data) into Bronze and verifies existence through the storage API. It confirms binary artifact handling, not just JSON handling. This is necessary because Bronze stores mixed artifact types, including images and other non-JSON outputs.
 - **test_bronze_ingestion_validates_dataset:** This test loads the bundled dummy training dataset through the dataset loader and checks required structural fields (`dataset_id`,

examples, room types, prompts, required assets). It verifies ingestion boundary validation before training workflows consume the dataset. This is necessary to fail early on malformed training inputs rather than surfacing errors mid-pipeline.

3. **tests/test_bronze_ingestion_distributed.py** This file extends Bronze validation to the distributed/lake-oriented training context, emphasizing run-scoped training prefixes and provenance artifacts. Although unit-scoped, it models the key layouts and artifact expectations used when training outputs are mirrored to lake storage. These tests are necessary because distributed prefix conventions are brittle and easy to regress when orchestration code changes.
 - **test_bronze_ingestion_single_scene**: This test writes one scene program into a run-scoped Bronze training prefix and verifies listing returns exactly that file with expected naming. It validates minimal ingestion behavior for a single artifact. This is necessary because even single-artifact runs must preserve deterministic key structure for traceability.
 - **test_bronze_ingestion_multiple_scenes**: This test writes multiple scene records under one run prefix and validates list cardinality. It ensures Bronze ingestion scales from one to several artifacts without path/listing regressions. This is necessary for realistic training runs that emit many per-example outputs.
 - **test_bronze_ingestion_validates_env**: This test checks that `get_store` returns a valid store under local app config and that Bronze writes succeed through that abstraction. It validates environment wiring between app config and storage backend selection. This is necessary because ingestion reliability depends on correct backend resolution before any artifacts are emitted.
 - **test_bronze_ingestion_run_manifest**: This test creates and finalizes a run manifest, stores it in Bronze, and validates persisted provenance fields (`run_id`, `status`, `run_type`). It confirms manifest persistence semantics in the Bronze layer. This is necessary because manifests are the primary audit trail for run-level reproducibility and failure analysis.
4. **tests/test_catalog_edge_cases.py** These tests lock in that behavior at the boundaries the module is meant to handle: strong keyword hits, ambiguous prompts, room-scoped asset extraction, empty keyword matches, the global supported-asset set, deduplication, and fallback to room defaults when inputs are invalid or unsupported.
 - **test_infer_room_type_prefers_matching_keywords**: Ensures that when the prompt clearly contains room-related cues (e.g. `dining`), `infer_room_type` resolves to the expected canonical id (`dining_room`) rather than a generic default. This guards the main “happy path” for room selection.
 - **test_infer_room_type_defaults_to_living_room_when_prompt_is_ambiguous**: Ensures that when the prompt does not strongly indicate a specific room, the implementation returns a stable default (`living_room`) instead of failing or returning an arbitrary type. That matters for pipelines that always need a valid room string.
 - **test_assets_mentioned_in_prompt_filters_assets_by_room_type**: Ensures that words in the prompt are only mapped to assets that are allowed for the given `room_type` (e.g. `bedroom`): unrelated mentions like “sofa” are not pulled in if the catalog ties them to another room. This prevents cross-room asset leakage when the user names furniture casually.

- **test_assets_mentioned_in_prompt_uses_room_defaults_when_nothing_matches:** Ensures that when no catalog keywords match for that room, the function still returns a sensible default asset list for the room (e.g. bedroom defaults) rather than an empty list. Empty lists could break or degrade downstream steps that expect at least baseline props.
- **test_all_supported_assets_returns_expected_flat_set:** Freezes the canonical set of supported asset identifiers returned by `all_supported_assets()`. Any change to that tuple is an API/catalog contract change; the test makes those edits visible and avoids silent drift in training or UI that lists supported types.
- **test_normalize_assets_deduplicates_and_falls_back_for_unsupported_inputs:** Ensures `normalize_assets` removes duplicates while preserving order, and that when the requested assets are invalid or wrong for the room (e.g. sofa, bed for dining_room), the function replaces them with the room's default asset bundle instead of passing junk through or returning nothing. That is the last normalization step before generation, so this branch is important for robustness.

5. tests/test_cli_edge_cases.py

- **test_run_or_exit_stage_pending_returns_exit1:** Validates that `StagePendingError` triggers a controlled CLI failure with exit code 1. Ensures stable exit semantics for automation.
- **test_run_or_exit_generic_exception_returns_exit1:** Confirms generic exceptions result in deterministic non-zero exit codes, maintaining a stable command-line error boundary.
- **test_cli_smoke_train_and_inference_commands:** Validates command registration and argument parsing for `train-hypotheses` and `run-inference` via monkeypatching.

6. tests/test_cli_env_wiring.py

- **test_train_and_inference_env_dev_load_store:** This test verifies that when `train-hypotheses` and `run-inference` are invoked with `-env dev`, the application loads configuration, obtains a storage backend, and attaches the same store instance to both service requests. It is necessary because environment-backed runs must consistently inject lake storage; a regression would cause training or inference to run without persistence or with inconsistent backends.
- **test_train_without_env_keeps_store_none:** This test verifies that `train-hypotheses` invoked without `-env` leaves store unset on the training request. It is necessary because local workflows rely on explicit opt-in for remote or layered storage; unintended store injection would change behavior for users who omit `-env`.
- **test_launch_aws_wait_invokes_runner_launch_and_wait:** This test verifies that `launch-aws` with `-wait` constructs the correct job specification from `-cli-args`, calls `launch`, and then calls `wait` with the returned instance identifier. It is necessary because the wait path is a distinct control flow; failure would strand jobs without monitoring or block incorrectly.

7. tests/test_coverage_gaps.py

- **test_stage_pending_raises_stage_pending_error:** This test verifies that `stage_pending` raises `StagePendingError` with an appropriate message for a designated stage. It is necessary because downstream code must distinguish “not implemented yet” from other failures when pipeline stages are stubbed.
- **test_load_app_config_env_none_uses_env_var:** This test verifies that `load_app_config(None)` respects the `COMPOS3D_ENV` environment variable. It is necessary because tooling and scripts may omit an explicit environment argument; behavior must remain deterministic and documented.
- **test_supported_room_types_returns_keys_tuple:** This test verifies that `supported_room_types()` enumerates exactly the keys of `SUPPORTED_ASSETS_BY_ROOM`. It is necessary because room catalogs and asset policies must stay aligned; drift would cause invalid room types or inconsistent UI and generation defaults.
- **test_load_experiment_config_vlm_defaults_provider_to_generator:** This test verifies that a VLM critic configuration with an empty provider inherits the generator provider when loading experiment config. It is necessary because partial configs should not leave the critic in an unusable state relative to the chosen generator.
- **test_cli_reference_generate_splits_tasks:** This test verifies that the `reference-generate` command parses `-tasks` into a tuple and passes `-minimal` through to the service. It is necessary because incorrect parsing would change procedural reference generation behavior without obvious runtime errors.
- **test_cli_build_scene_and_feature_extract_routes:** This test verifies that `build-scene` forwards flags such as `-no-video` and `-save-blend`, and that `feature-extract` passes input and output paths to the underlying functions. It is necessary because the CLI is the primary user entry point; mis-wiring would silently drop options.
- **test_cli_launch_aws_instance_type_override_and_main_guard:** This test verifies the `-instance-type` override on `launch-aws` and exercises the module’s `__main__` entry (help and exit). It is necessary because override logic and the executable entry point are easy to break without dedicated coverage.
- **test_storage_get_store_s3_success:** This test verifies that an S3-backed `AppConfig` yields a `MultiLayerS3Store` from `get_store`. It is necessary because the factory must select the correct implementation for remote lakes.
- **test_multilayer_s3_store_exists_false_on_client_error:** This test verifies that S3 client errors on `head_object` are treated as non-existence for `exists`. It is necessary because transient or permission errors must not be conflated with definitive existence without a defined policy; this encodes the chosen policy.
- **test_manifest_git_info_remote_url_missing_and_packages_unknown:** This test verifies manifest creation when git remote URL is missing and package versions cannot be resolved. It is necessary because CI and minimal clones often lack full git metadata; manifests must still be produced.
- **test_manifest_git_info_total_failure_returns_none:** This test verifies that total failure of git subprocess calls results in `git.info` being absent rather than crashing manifest creation. It is necessary for robustness in restricted environments.
- **test_procedural_service_backend_smoke_generate_reference_build_scene_and_feature_extraction:** This test verifies procedural service paths—backend smoke, reference generation, build scene variants, and stage-pending feature extraction—using faked script execution and

filesystem outputs. It is necessary because these branches involve multiple flags and manifest writes that integration tests may not exercise exhaustively.

- **test_runner_find_gin_config_root_from_sibling:** This test verifies discovery of gin configuration roots via sibling directory layout and correct merging of PYTHONPATH in the runner environment. It is necessary for Infinigen-style project layouts and subprocess environments.
- **test_critic_remaining_branches_and_empty_aggregate:** This test verifies aggregate scoring on empty inputs, note normalization, image format selection, prompts with reference examples, invalid critic factory configurations, and VLM evaluation without images. It is necessary because these are defensive and error branches in the evaluation layer.
- **test_hypothesis_loop_render_success_and_only_best_hypothesis:** This test verifies successful rendering delegation and the only_best_hypothesis branch during regeneration. It is necessary because bandit-style loops depend on these branches for correct hypothesis bank updates.
- **test_hypothesis_engine_mirror_inference_and_store_fields:** This test verifies lake mirroring of inference artifacts and manifest fields when a store is supplied, using a fake store. It is necessary because data lake integration must write expected keys and expose URIs in the manifest.
- **test_scene_llm_bedrock_paths_and_build_scene_llm_branches:** This test verifies Bedrock error mapping, empty hypothesis handling under repair focus, successful scene program generation, and build_scene_llm provider resolution. It is necessary because the LLM layer has multiple providers and failure modes that must remain stable.
- **test_ec2_runner_remaining_branches:** This test verifies EC2 launch with networking, spot options, fixed AMI, wait timeout, and AMI resolution failure when no image is found, using fake boto3 clients. It is necessary because cloud launch code contains many branches that cannot be covered by live AWS tests in CI.
- **test_multilayer_s3_store_read_json_and_exists_true:** This test verifies successful JSON reads and positive existence checks against a fake S3 client. It is necessary to cover the happy path for bronze/silver/gold reads without real buckets.
- **test_hypothesis_engine_train_config_path_room_filter_and_inference_render_scene:** This test verifies that training respects room_types filtering from config and that inference with render_scene=True exercises the rendering path with faked dependencies. It is necessary because configuration-driven filtering and optional rendering are distinct code paths in the engine.
- **test_runner_run_script_non_capture_success_and_failure:** This test verifies subprocess invocation without output capture on success and failure handling with captured stderr when the process exits non-zero. It is necessary because the runner supports multiple invocation modes used by procedural scripts.
- **test_scene_llm_normalizers_cover_error_branches:** This test verifies JSON extraction, style and count heuristics, and strict validation for hypotheses, constraints, assets, and full scene payloads, including deduplication and mock repair hypotheses. It is necessary because malformed LLM output must be rejected or normalized consistently at the system boundary.
- **test_ec2_runner_resolve_ami_and_wait_paths:** This test verifies AMI resolution among multiple images, launch, polling until termination, and instance termination, using fakes.

It is necessary to cover AMI selection and lifecycle logic without incurring AWS cost or flakiness.

8. `tests/test_critic_edge_cases.py`

- **`test_critic_handles_empty_assets_with_low_validity_and_note`**: Ensures empty predicted assets are penalized with low validity to preserve metric quality in common failure modes.
- **`test_critic_marks_mismatched_room_type_against_reference`**: Verifies that room-type mismatches yield `room_match == 0.0`, enforcing core semantic requirements.
- **`test_critic_uses_unique_assets_for_precision_recall_calculation`**: Confirms duplicate assets do not inflate metrics by using set-based comparisons, protecting metric integrity.
- **`test_critic_invalid_room_type_forces_zero_validity`**: Validates that unsupported room types are hard-penalized to protect ontology constraints.
- **`test_critic_contradictory_constraints_do_not_crash_heuristic`**: Ensures robustness against inconsistent instructions, preventing crashes and maintaining bounded outputs under noisy input.

9. `tests/test_dataset_loader_validation.py`

- **`test_training_dataset_rejects_missing_required_fields`**: Ensures malformed dataset examples are rejected early to prevent downstream failures in expensive stages.
- **`test_training_dataset_rejects_missing_prompt`**: Verifies that missing prompts trigger validation errors, protecting the data contract foundation.
- **`test_training_dataset_empty_prompt_currently_allowed_model_behavior`**: Documents that empty prompts are currently accepted; serves as a contract baseline for future schema changes.
- **`test_training_dataset_room_type_is_not_enum_restricted`**: Documents that room type is currently a free string, flagging potential issues if validation semantics tighten in the future.

10. `tests/test_experiment_config_edge_cases.py`

- **`test_load_experiment_config_none_returns_independent_default_copy`**: This test verifies that `load_experiment_config(None)` returns a valid default experiment configuration (mock generator, heuristic critic) and that successive calls yield independent objects so mutating one does not alter the other. It is necessary because shared mutable defaults would cause subtle cross-run corruption when callers adjust training or critic settings in memory.
- **`test_load_experiment_config_autofills_bedrock_generator_model_id`**: This test verifies that a generator specifying provider: "bedrock" without an explicit `model_id` receives the configured default text model identifier. It is necessary because incomplete JSON should still yield a runnable Bedrock configuration rather than failing late or passing an invalid model reference.
- **`test_load_experiment_config_vlm_sets_bedrock_defaults`**: This test verifies that a VLM critic with Bedrock as provider receives the default vision model identifier (and

consistent provider) when omitted from the file. It is necessary to ensure VLM mode remains operational under minimal critic configuration, symmetric with generator aut-offill.

- **test_load_experiment_config_vlm_rejects_non_bedrock_provider:** This test verifies that VLM critic mode with a non-Bedrock provider raises a clear `ValueError`. It is necessary because unsupported combinations should fail at load time with an explicit contract, rather than at runtime inside the critic or Bedrock client.

11. tests/test_filter_and_weight_inference.py

- **test_mock_llm_filter_relevant_hypotheses_prefers_prompt_overlap:** This test verifies that `MockSceneLLM.filter_relevant_hypotheses` returns hypotheses whose wording aligns with the user prompt (e.g. dining table and chairs) and excludes weakly related candidates. It is necessary because the mock must approximate sensible filtering for offline tests and for development without Bedrock.
- **test_bedrock_filter_relevant_hypotheses_bad_payload_raises:** This test verifies that when Bedrock-style filtering returns a malformed `relevant_hypotheses` field (not a list), the implementation raises `StructuredOutputError` with an informative message. It is necessary to enforce structured-output contracts and to surface model formatting errors deterministically.
- **test_weighted_vote_scene_program_aggregates_assets_and_style:** This test verifies that `_weighted_vote_scene_program` merges multiple candidate `SceneProgram` instances using hypothesis weights, preserves style from higher-accuracy hypotheses, merges assets (including counts such as four chairs), and retains additional assets from other programs when appropriate. It is necessary because this aggregation is the core of the filter-and-weight inference path; regressions would directly distort generated scenes.
- **test_engine_filter_and_weight_helper_branches:** This test verifies auxiliary behavior: inference strategy strings are normalized case-insensitively; empty inputs to `_best_accuracy_hypotheses` yield `None`; zero accuracy still yields a minimum vote weight of 1.0; `_fallback_asset_spec` supplies a defined placement for requested asset types; invalid strategy strings raise `ValueError` mentioning `inference_strategy`. It is necessary because these helpers constrain edge cases in strategy parsing and voting math.
- **test_filter_hypotheses_for_inference_uses_heuristic_fallback_on_errors:** This test verifies that when `filter_relevant_hypotheses` raises, `_filter_hypotheses_for_inference` falls back to a heuristic that ranks hypotheses by lexical overlap with the prompt. It is necessary because LLM calls are unreliable; inference must degrade gracefully rather than abort.
- **test_weighted_vote_scene_program_uses_fallback_asset_and_majority_threshold:** This test verifies that when the prompt requests an asset (e.g. chair) missing from candidates, a fallback asset specification is introduced, and that voting thresholds still allow inclusion of assets such as a rug according to record weights. It is necessary to cover gap-filling and threshold logic that would otherwise leave user-requested content unrepresented.
- **test_filter_and_weight_falls_back_to_best_accuracy_when_filter_empty:** This test verifies that an empty filter result triggers selection by best accuracy, and that downstream `generate_scene_program` receives the corresponding hypothesis text so the produced program reflects that choice (e.g. chair vs lamp). It is necessary because an empty

LLM filter must not stall the pipeline; accuracy-based fallback must remain wired correctly.

- **test_filter_and_weight_skips_failed_candidates_and_falls_back_to_joint_generation:** This test verifies that when per-hypothesis generation fails for single-hypothesis calls but succeeds for the joint hypothesis list, the pipeline selects both records and returns a program built from the joint call. It is necessary to validate retry/fallback logic that avoids losing valid hypotheses after partial failures.
 - **test_run_vertical_inference_filter_and_weight_end_to_end:** This test verifies that `run_vertical_inference` with `inference_strategy="filter_and_weight"` loads a bank from disk, invokes a stubbed LLM and critic, writes outputs, and produces a manifest whose strategy, selected hypothesis ids, and serialized scene program match filter-and-weight behavior. It is necessary because it exercises the public entry point and file I/O path, not only private helpers, ensuring CLI and service callers see consistent results.
12. **tests/test_hypogenic.py** This file is a unit test suite for the HypoGeniC/UCB hypothesis-bank mechanics: ranking, exploration behavior, update bookkeeping, persistence, and basic bank statistics. It targets the selection/update mathematics that govern learning quality over time. These tests are necessary because subtle regressions in UCB logic can degrade model behavior without causing explicit runtime failures.
- **test_hypothesis_record_creation:** This test verifies that `HypothesisRecord` instances can be created and serialized with expected defaults. It validates model construction integrity. This is necessary because record schema stability is foundational for bank persistence and downstream processing.
 - **test_ucb_score_unvisited_is_infinite:** This test asserts that unvisited hypotheses receive infinite UCB score and therefore exploration priority. It validates the exploration-first property for unseen candidates. This is necessary to prevent premature exploitation that can reduce hypothesis diversity and learning efficiency.
 - **test_ucb_score_decreases_with_visits:** This test checks that, with constant accuracy, UCB scores decrease as visit count increases (shrinking exploration term). It validates monotonic exploration decay. This is necessary for the intended exploration-exploitation balance over time.
 - **test_hypothesis_selection_prefers_unvisited:** This test constructs a mixed pool and verifies the unvisited hypothesis is selected first by UCB ranking. It validates practical selection behavior, not just formula-level expectations. This is necessary to ensure selection policy implementation matches theoretical intent.
 - **test_hypothesis_update_fields:** This test simulates a successful evaluation update and validates incremented visits/successes and reward/accuracy updates. It confirms state transitions after scoring. This is necessary because incorrect bookkeeping corrupts future ranking and training dynamics.
 - **test_hypothesis_bank_save_and_load:** This test serializes a bank to JSON and reloads it back into typed records, validating round-trip integrity. It checks persistence compatibility. This is necessary for resumable training and reproducible inference from saved banks.
 - **test_hypothesis_bank_statistics:** This test computes aggregate bank statistics (total visits and per-room grouping) from representative records. It validates basic analytic

consistency over record fields. This is necessary because many training/evaluation summaries depend on these aggregates being correct.

- **test_top_k_selection_by_ucb:** This test validates top-k selection behavior under mixed profiles, including multiple unvisited hypotheses with infinite UCB scores. It ensures ranking/selection logic handles edge cases correctly. This is necessary because top-k extraction is a core operational path for hypothesis usage.

13. tests/test_hypothesis_engine_edge_cases.py

- **test_select_hypotheses_for_inference_ties_and_topk:** This test verifies that inference-time hypothesis selection respects room filtering, ranking logic, and top_k truncation under tied or near-tied candidates. The selection function is central to output quality because it determines which hypotheses condition generation. This test is necessary to guard against subtle ranking regressions. A failure indicates inference may select weaker or irrelevant hypotheses.
- **test_training_and_loop_config_helpers:** This test validates helper transformations from raw argument values into TrainingConfig and then into HypothesisLoopConfig. The conversion pipeline is a critical control plane for experiments and ablations. It is necessary because many runtime behaviors depend on these mappings. A failure indicates parameter drift risk.
- **test_mirror_training_and_inference_with_missing_optional_files:** This test checks that lake mirroring routines handle missing optional artifacts gracefully while still writing mandatory metadata. The mirroring logic is intentionally conditional for artifacts such as metrics, snapshots, and render outputs. This test is needed to ensure partial outputs do not cause full pipeline failures.
- **test_train_and_inference_manifest_failure_paths:** This test exercises manifest finalization behavior when storage/mirroring fails during both training and inference paths. The key objective is validating failure-mode robustness: errors should propagate, while run manifest writes in finally remain attempted. This is necessary for observability and postmortem traceability.
- **test_evaluate_prediction_dir_paths_and_missing:** This test validates all major branches of evaluate_prediction_dir: standard predictions.jsonl path, fallback inference_manifest + critic_score path, and explicit missing-input failure. It is necessary because evaluation is frequently run on heterogeneous artifact sets.

14. tests/test_hypothesis_loop_edge_cases.py

- **test_maybe_regenerate_early_returns:** This test verifies multiple early-return conditions in _maybe_regenerate, including disabled repair mode, fixed-hypothesis baseline mode, non-positive target size, and insufficient pending examples. These guards prevent unnecessary regeneration work and preserve intended ablation semantics. A failure indicates regeneration may trigger when it should not.
- **test_wrong_threshold_edge_cases:** This test validates edge behavior of _wrong_threshold, specifically zero selected hypotheses and non-positive scaling mode. Threshold correctness directly controls when failures trigger repair logic. This test is necessary to ensure deterministic behavior in edge conditions.

- **test_replace_room_bank_dedup_and_cap:** This test confirms `_replace_room_bank` deduplicates near-identical hypothesis text and enforces maximum bank size per room. Bank hygiene is fundamental for stable hypothesis selection and avoiding redundant records. A failure indicates risk of bank bloat and reduced diversity.
- **test_render_exception_path_returns_empty:** This test verifies that renderer exceptions are handled gracefully and converted into an empty image-path result instead of crashing the training loop. Rendering is an optional but failure-prone dependency, so graceful degradation is essential.
- **test_score_handles_critic_unavailable:** This test validates fallback behavior when the critic is unavailable: `_score` should return a structured zero-score payload with explicit `critic_mode="unavailable"` and diagnostic notes. This behavior is necessary to keep training robust during temporary critic outages.
- **test_baseline_mode_no_hypotheses_branch:** This test checks that baseline mode `no_hypotheses` removes hypothesis conditioning in both hypothesis evaluation and final prediction construction. It is necessary because baseline-mode correctness underpins fair ablation comparisons.

15. `tests/test_hypothesis_update_rules.py`

- **test_update_record_updates_running_mean_accuracy_and_counts:** This test verifies that `_update_record` correctly updates visit counts, success counts, running mean score, and accuracy for successful evaluations. These fields drive downstream ranking and reward updates. A failure indicates record statistics are unreliable and can skew hypothesis selection.
- **test_update_record_adds_failure_tag_when_below_threshold:** This test ensures failed evaluations append failure tags without incorrectly incrementing success counters. Failure tagging influences repair context and diagnostics, making correctness important for adaptive behavior.
- **test_compute_reward_bounds_and_zero_visits_behavior:** This test validates reward function edge cases: zero visits returns 0.0, early sample behavior returns plain accuracy, and exploration bonus formula is applied for later samples. A failure indicates reward drift and potentially unstable hypothesis ranking.
- **test_sort_records_tie_breaking_is_deterministic:** This test verifies deterministic tie-breaking in record sorting when major metrics are equal. Determinism is important for reproducibility and debugging consistency. A failure indicates unstable ranking order and reduced experiment repeatability.

16. `tests/test_local_store_errors.py`

- **test_read_json_missing_file_raises:** This test validates that reading a missing JSON object raises `FileNotFoundError` explicitly. Clear error typing is critical for callers that branch on missing-artifact behavior. A failure indicates weak storage error signaling and harder upstream recovery logic.
- **test_read_json_invalid_json_raises:** This test ensures invalid JSON content produces a `JSONDecodeError`, preserving parsing transparency. This is necessary because corrupted artifacts should fail loudly and diagnostically. A failure indicates parse failures may be masked.

- **test_put_json_overwrite_semantics:** This test verifies overwrite behavior when writing the same path multiple times. Overwrite semantics are frequently relied upon in iterative pipelines. A failure indicates artifact update behavior is inconsistent.
- **test_list_prefix_nested_paths_and_missing_prefix:** This test validates recursive listing for nested keys and correct empty result for missing prefixes. This test is needed because listing APIs are foundational to lake inspection and aggregation steps.

17. tests/test_manifest_schema.py

- **test_create_manifest_has_required_metadata:** This test verifies that manifest creation includes required provenance and runtime metadata (run_id, run_type, status, python version, package versions, timezone-aware start time). Manifest completeness is critical for reproducibility and auditability.
- **test_finalize_manifest_success_sets_completion_fields:** This test confirms successful finalization populates completed_at, duration_seconds, status, and output URIs. Finalization correctness is essential for monitoring and downstream analytics.
- **test_finalize_manifest_failed_status_keeps_error_message:** This test ensures failed finalization preserves error_message while setting failure status. Failure-context persistence is essential for operational triage.
- **test_manifest_timestamp_consistency_after_finalize:** This test validates temporal consistency: completion timestamp must be after start timestamp and duration must be non-negative and plausible. A failure indicates unreliable run timing metrics.

18. tests/test_multibucket_s3_store.py

- **test_multilayer_s3_store_routes_writes_to_expected_buckets:** This test verifies that put_json / put_bytes send objects to the bronze, silver, or gold bucket according to whether the key path contains bronze, silver, or gold (including mixed casing for the silver example), and that returned URIs include the configured prefix and bucket name. It is necessary because multi-bucket routing is the core responsibility of MultiLayerS3Store; incorrect routing would write to the wrong lake layer or produce unusable URIs.
- **test_multilayer_s3_store_defaults_ambiguous_paths_to_bronze:** This test verifies that keys without an explicit layer prefix are written to the bronze bucket. It is necessary because ambiguous keys must have a defined default; silent mis-routing to silver or gold would corrupt lake semantics.
- **test_multilayer_s3_store_list_prefix_collects_paginated_results:** This test verifies that list_prefix follows the correct format, issues follow-up list_objects_v2 calls, merges keys from all pages, and lists against the bucket implied by the prefix (here silver). It is necessary because real buckets often paginate; incomplete listing would hide objects and break downstream jobs that scan the lake.
- **test_multilayer_s3_store_exists_returns_false_on_client_error:** This test verifies that exists uses head_object on the correct bucket for a gold-prefixed key and returns False when the client is fake (e.g. missing object). It is necessary to encode the chosen policy for errors (treat as non-existent) and to ensure existence checks target the right bucket for layered paths.

19. tests/test_procedural_service_runner_edge_cases.py

- **test_generate_reference_scene_invalid_room_type_raises:** This test verifies that `generate_reference_scene` rejects unsupported room types with a clear `ValueError`. Input validation at this boundary is critical because downstream procedural generation is expensive and should not execute on invalid semantic categories. The test is necessary to ensure invalid requests fail fast and predictably. A failure indicates weak guardrails and increased likelihood of expensive no-op or undefined procedural runs.
- **test_run_backend_smoke_missing_render_raises:** This test validates post-run artifact checks in the backend smoke path. Even if the subprocess returns a nominal success result, the expected render artifact must exist; otherwise a runtime error should be raised. This protects against partial or silent subprocess failures where return codes are not sufficient evidence of success. A failure indicates artifact integrity checks are missing or ineffective.
- **test_build_scene_missing_input_file_surfaces_subprocess_error:** This test ensures that build-scene failures (for example, missing scene-program input) are surfaced as explicit runtime errors rather than swallowed. The procedural build step is a high-cost integration path where clear failure messaging is essential for diagnosis. The test is necessary to preserve transparent failure propagation from runner to service layer. A failure indicates error context may be lost and operational debugging becomes harder.
- **test_runner_run_script_nonzero_exit_raises:** This test confirms `run_script` raises `RuntimeError` when subprocess exits non-zero, including failure context in the message. This behavior is fundamental for command orchestration correctness because non-zero exits must be interpreted as hard failures. The test is needed to prevent silent continuation after failed subprocesses. A failure indicates orchestration layers may incorrectly treat failed procedural jobs as successful.
- **test_runner_run_script_capture_output_success:** This test validates the success path when subprocess output capture is enabled, ensuring expected keys (`exit_code`, `stdout`) are present and consistent. It is necessary because captured output is often used for diagnostics and reporting in test/CI workflows. A failure indicates result payload contract instability and reduced debuggability.

20. `tests/test_scene_llm_edge_cases.py`

- **test_build_scene_llm_provider_selection_and_invalid:** This test verifies provider dispatch for both string and config-based inputs and checks unsupported provider rejection. Provider routing is foundational for model backend selection and experiment reproducibility. The test is necessary to detect regressions in dispatch logic and validation behavior. A failure indicates backend selection ambiguity and possible accidental fallback to unintended providers.
- **test_bedrock_run_json_prompt_invalid_output_raises_structured_error:** This test ensures malformed Bedrock response shapes are mapped to `StructuredOutputError`. Model-output parsing failures are expected in real integrations and must fail with domain-specific clarity. The test is needed to guarantee robust handling of invalid provider payloads. A failure indicates brittle parsing and potential propagation of opaque key/index errors.
- **test_bedrock_run_json_prompt_credential_error_maps_to_unavailable:** This test validates credential/session-expiry mapping to `LLMUnavailableError` with reauthentication guidance. It is necessary because credential issues are common and recoverable, and

callers need consistent error semantics. A failure indicates users/operators may receive low-signal infrastructure exceptions.

- **test_generate_hypotheses_bad_payload_raises:** This test checks that hypothesis generation rejects non-list hypotheses payloads from model output. Structured output enforcement is essential to keep downstream loop logic type-safe and deterministic. The test is necessary to prevent malformed model outputs from entering hypothesis banks. A failure indicates weak schema enforcement at generation boundaries.
- **test_generate_scene_program_normalization_failure_raises:** This test verifies normalization guards for scene-program payload consistency, specifically room-type mismatch behavior. It is necessary because semantic mismatch in generated scene meta-data undermines correctness and evaluation validity. A failure indicates normalization contracts can be bypassed by malformed model responses.
- **test_determinism_controls_forwarded_inference_config:** This test confirms configured generation controls (temperature, maxTokens) are actually forwarded to Bedrock calls. This is required for reproducibility and controlled experimentation. A failure indicates runtime request behavior may diverge from declared configuration, reducing experiment reliability.

21. tests/test_scene_program_validation.py

- **test_scene_program_requires_prompt_and_room_type:** This test verifies that core required fields are enforced for SceneProgram. It is necessary because prompt and room type define the semantic target and missing either should fail immediately. A failure indicates schema weakening that can allow invalid objects into generation/evaluation pipelines.
- **test_asset_spec_count_must_be_positive:** This test checks numeric constraints on AssetSpec.count, ensuring non-positive counts are rejected. It is necessary to maintain physically meaningful scene descriptions and prevent invalid object specifications. A failure indicates numeric validation drift and potential downstream rendering/scoring anomalies.
- **test_asset_spec_rejects_invalid_count_type:** This test ensures non-numeric count values trigger validation failure. Type enforcement is important because count fields are used in arithmetic and aggregation logic. A failure indicates increased risk of runtime type errors after ingestion.
- **test_constraint_spec_requires_text_field:** This test validates that constraints contain required textual content. Constraint quality affects explainability and policy checks in scene interpretation. A failure indicates malformed constraints may pass validation and degrade later logic.
- **test_render_spec_defaults_are_applied:** This test verifies default initialization behavior for RenderSpec.mode. Default stability is important for callers that rely on implicit render behavior. A failure indicates backward compatibility risk for code paths that omit explicit render spec.
- **test_scene_program_accepts_malformed_constraint_shape_as_validation_error:** This test confirms malformed nested constraint objects are rejected at model validation time. It is necessary to prevent structurally invalid nested payloads from propagating silently. A failure indicates nested validation gaps.

- **test_scene_program_current_model_does_not_enforce_room_enum:** This behavior-contract test documents that `room.type` is currently a free-form string. It is necessary to make schema semantics explicit and detect future tightening as an intentional change. A failure indicates contract change and potential need for dataset migration or test fixture updates.

22. `tests/test_selection_strategies.py`

- **test_selection_empty_bank_returns_empty:** This test verifies safe behavior when the room-specific hypothesis bank is empty. Empty-bank handling is a common edge case during initialization or filtered runs. The test is necessary to prevent crashes and ensure deterministic no-selection behavior. A failure indicates control-flow instability in early or sparse training phases.
- **test_selection_ucb_uses_reward_sorting:** This test validates selection behavior for the UCB/default strategy path, ensuring ordering aligns with reward-based ranking semantics. It is necessary because selection policy strongly influences training dynamics and hypothesis reinforcement. A failure indicates policy drift and altered learning behavior.
- **test_selection_greedy_uses_accuracy_then_mean_score_then_visits:** This test verifies greedy strategy ranking tuple behavior. It is important for ablation correctness, where strategy-specific behavior must remain distinct and intentional. A failure indicates strategy implementation drift or tie-break inconsistencies.
- **test_selection_random_respects_k_and_room_filter:** This test ensures random strategy still respects cardinality (`k`) and room filtering constraints. Even random policy must satisfy hard selection invariants. A failure indicates sampling may leak cross-room hypotheses or violate expected sample size.

23. `tests/test_service_request_validation.py`

- **test_training_request_defaults_and_forwarding:** This test verifies that `TrainingRequest` defaults are populated correctly and forwarded unchanged into the training engine endpoint. It is necessary because service wrappers are API boundaries and should not mutate or drop user intent. A failure indicates request-contract drift between service and engine layers.
- **test_inference_request_defaults_and_forwarding:** This test checks default inference knobs (including render defaults) and argument forwarding into inference engine calls. It is needed to preserve predictable service-layer behavior and prevent silent default changes. A failure indicates wrapper-level configuration mismatches that can alter output behavior unexpectedly.
- **test_evaluate_request_forwarding:** This test validates that evaluation service wrapper passes correct paths to engine evaluation logic. It is necessary to guarantee minimal, transparent service orchestration behavior. A failure indicates path wiring regressions and potential mis-targeted artifact evaluation.

24. `tests/test_spatiallm_conversion.py`

- **test_canonicalize_source_room_type_maps_supported_variants:** Verifies that source labels like "master bedroom" map to canonical IDs like "bedroom." It is necessary because training ingestion depends on stable room typing; ambiguous labels must be filtered or mapped correctly.

- **test_parse_layout_asset_counts_maps_labels_and_windows:** Verifies that SpatialLM layout text (Bbox strings) normalizes to counted asset types. It is necessary because downstream selection is driven by these counts; parser drift would corrupt training inputs.
- **test_select_required_assets_prefers_room_priority:** Verifies that `select_required_assets` returns a priority-ordered subset of assets. It is necessary to match catalog expectations and keep prompts aligned with room semantics.
- **test_build_prompt_uses_room_specific_nouns:** Verifies that bedroom prompts use "nightstand" for `table_top`, etc. It is necessary because prompts must read naturally per room and reflect specific noun mapping rules.
- **test_spatiallm_helper_functions_cover_edge_cases:** Verifies string utilities like pluralization and CSV loading. It is necessary because these helpers are reused across conversion; irregular plurals (e.g., "boxes") must remain stable.
- **test_select_required_assets_and_candidate_sort_key_edge_cases:** Verifies that rooms are rejected when mandatory anchors are missing (e.g., dining room without a table). It is necessary to ensure the generated dataset contains only high-quality, usable examples.
- **test_collect_spatiallm_candidates_and_payload_generation:** Verifies end-to-end conversion logic with a patched row iterator. It is necessary as a regression harness for the main conversion pipeline without requiring real JSONL source files.
- **test_spatiallm_loader_branches_and_candidate_filters:** Verifies that the richer candidate (more assets) is selected when duplicates exist. It is necessary to cover loader integration and quality-based filtering across data splits.

25. `tests/test_spatiallm_conversion_edge_cases.py`

- **test_spatiallm_helper_branches_cover_selection_and_phrase_shapes:** Verifies additional noun mappings and asset descriptions for large counts ("several"). It is necessary because the main test file does not exhaustively cover every asset category.
- **test_spatiallm_metadata_and_row_loader_helpers:** Verifies CSV metadata loading from a temp directory and the Hugging Face loader contract. It is necessary to validate filesystem parsing and row-handling logic.
- **test_load_local_spatiallm_rows_raises_when_datasets_is_unavailable:** Verifies that a clear `RuntimeError` is raised if the optional `datasets` package is missing. It is necessary to provide helpful error messaging to users instead of obscure import traces.
- **test_collect_spatiallm_candidates_filters_duplicates_and_sorts:** Verifies multi-file ingestion and row-level filtering (e.g., skipping short conversations). It is necessary because real-world data is noisy; these rules are critical for final dataset quality.
- **test_candidate_sort_payload_write_and_summary:** Verifies stable selection order and nested directory creation during JSON export. It is necessary to lock serialization behavior and ensure exported training files are deterministically structured.

26. `tests/test_storage_and_config.py` This file is a unit test suite for configuration loading, storage backend selection, local store behavior, and canonical path utilities. It validates both happy-path and key error-path behavior, especially around S3 configuration requirements and layer separation invariants. These tests are necessary because config/storage failures are systemic and can break every stage in the pipeline.

- **TestAppConfig.test_default_is_local:** Validates that default app config resolves to local environment and local backend. Necessary to ensure safe developer defaults and predictable no-cloud startup behavior.
- **TestAppConfig.test_local_env_yaml:** Loads local env config and verifies local backend settings. Necessary to validate environment-specific config resolution from YAML.
- **TestAppConfig.test_dev_env_yaml:** Loads dev config and verifies S3 backend plus expected bucket names. Necessary to ensure non-local environments are wired to intended cloud resources.
- **TestAppConfig.test_staging_env_yaml:** Validates staging bucket resolution. Necessary for environment isolation and deployment correctness.
- **TestAppConfig.test_prod_env_yaml:** Validates production bucket and on-demand instance policy (`ec2_spot` is `False`). Necessary to enforce production safety/cost assumptions.
- **TestAppConfig.test_get_store_local:** Ensures local config returns a `LocalStore` instance. Necessary to guarantee backend factory correctness in local workflows.
- **TestAppConfig.test_get_store_s3_missing_buckets_raises:** Asserts misconfigured S3 backend raises a clear `ValueError` when required buckets are missing. Necessary for fail-fast behavior on invalid infrastructure configuration.
- **TestLocalStore.test_put_and_read_json:** Validates JSON write/read round-trip in local store. Necessary as baseline contract for metadata and structured artifact persistence.
- **TestLocalStore.test_put_and_read_bytes:** Validates binary write/read path in local store. Necessary because pipelines persist both structured and binary outputs.
- **TestLocalStore.test_list_prefix:** Checks prefix listing returns expected files under Bronze prefix only. Necessary for deterministic artifact discovery and layer-scoped scans.
- **TestLocalStore.test_exists:** Validates existence checks before/after object write. Necessary for defensive pipeline logic around optional/missing artifacts.
- **TestLocalStore.test_bronze_silver_gold_dirs_are_separate:** Writes same logical file name into all three layers and verifies retrieval isolation by layer. Necessary to preserve data-lake semantics and prevent accidental overwrite/leakage across Bronze/Silver/Gold.
- **TestLakePaths.test_training_paths:** Validates generated training prefixes for Bronze, Silver, and Gold naming conventions. Necessary to keep path contracts stable across modules.
- **TestLakePaths.test_inference_paths:** Validates generated inference prefixes. Necessary for consistent read/write addressing in inference and evaluation tooling.
- **TestLakePaths.test_utc_date_parts_format:** Checks UTC date-part string formatting (year/month/day widths). Necessary for deterministic partition keys and chronological organization standards.

27. tests/test_storage_paths.py

- **test_training_path_builders_are_stable:** This test verifies deterministic key construction for training bronze/silver/gold prefixes. Stable key schemas are essential for downstream tooling, analytics, and artifact retrieval. A failure indicates path contract change that may break existing storage consumers.

- **test_inference_path_builders_are_stable:** This test validates deterministic key construction for inference prefix helpers. It is necessary for consistent artifact organization and retrieval in inference pipelines. A failure indicates namespace drift and potential breakage in storage integrations.
- **test_utc_date_parts_zero_padded_and_utc_based:** This test confirms date partition helpers produce UTC-based, zero-padded components. Date partition consistency is critical for predictable storage layout and queryability across environments. A failure indicates partitioning inconsistency and possible data discoverability issues.

28. tests/test_training_renderer_edge_cases.py

- **test_make_training_renderer_returns_none_when_disabled:** This test verifies that when `RenderConfig.enabled` is false, `make_training_renderer` returns `None` instead of a callable renderer. It is necessary because the training loop must skip expensive render setup when rendering is turned off; returning a no-op or a partial renderer would waste resources or fail later in opaque ways.
- **test_make_training_renderer_writes_scene_program_and_filters_missing_paths:** This test verifies that when rendering is enabled, `make_training_renderer` returns a function that: uses a controlled temp layout, serializes the `SceneProgram` to `scene_program.json` under the run directory, invokes `build_scene` with a `BuildSceneRequest` whose `scene_program_path`, `resolution`, `view_samples`, `no_video=True` (training always disables video), and `save_blend` match the render config; and filters `rendered_views` so only paths that exist on disk are returned (dropping declared-but-missing PNGs). It also asserts cleanup via `rmtree` on the temp root. It is necessary because this helper is the bridge between training and the procedural backend: incorrect request wiring would break builds, forcing `no_video` incorrectly would slow training or fail pipelines, and missing-file filtering prevents downstream code from referencing nonexistent images after partial render failures.

29. tests/test_training_resume_and_wandb.py

- **test_training_resume_continues_from_saved_state:** This test verifies that after training raises mid-run (simulated failure immediately after persisting progress), `resume_state.json` exists on disk, and that starting a new loop with `resume=True` on the same `run_dir` completes with the same `num_predictions`, same metrics, and same final `hypothesis_bank.json` as a fresh run from scratch in a separate directory. It is necessary because long training jobs must be restartable without duplicating work or diverging from a clean run; regressions in persistence or resume logic would corrupt experiment reproducibility and operator trust.
- **test_training_logs_to_wandb_when_enabled:** This test verifies that when `LoggingConfig` enables W&B with a short log cadence, `train()` invokes the patched `wandb.init`, records scalar logs containing expected keys (e.g. combined overall metrics, bank snapshots, init and prediction previews, prediction animation, tabular metrics), logs artifacts that include `resume_state.json` and media manifest paths, calls `define_metric`, and ends with `finish()` on the run. It uses a fake renderer that produces minimal multi-view PNGs so media logging paths execute. It is necessary because observability hooks are easy to break silently; the test ensures enabled logging actually emits the intended metrics and artifacts without requiring a real W&B server or project.

5 Two Memorable Bugs

Walkthrough video of the two bugs with fixes and testing:

https://www.youtube.com/watch?v=guHTt4_2uts